Autor: Kacper Sosnowski

Tytuł projektu: STOPER

1. Cel projektu

Celem projektu było stworzenie stopera w języku VHDL, który umożliwia realizację następujących funkcjonalności:

- Start oraz zatrzymanie działania stopera,
- Zerowanie wyniku,
- Zapamiętanie maksymalnie trzech wyników,
- Wyświetlanie wyników w formacie MM:SS

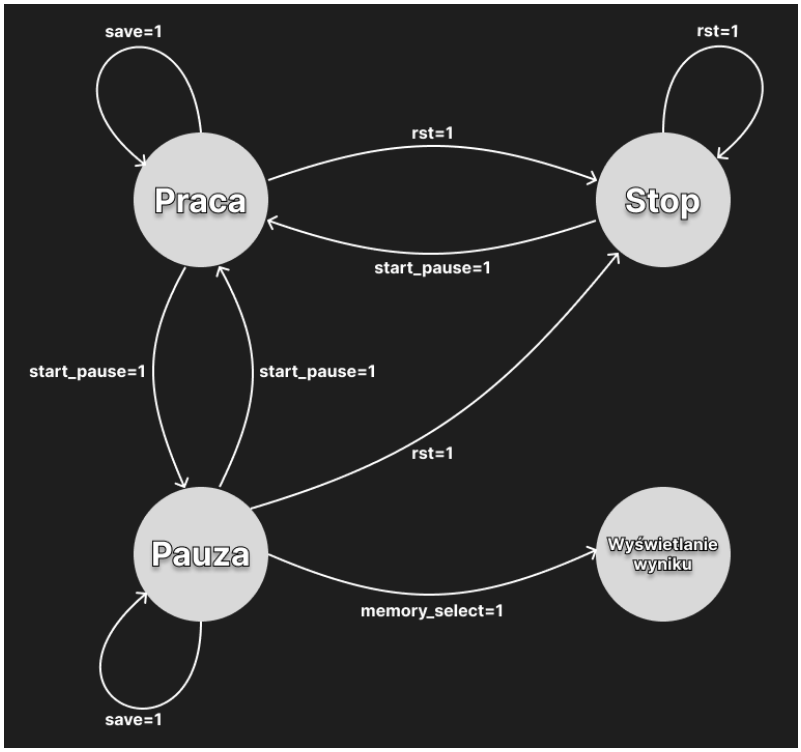
2. Założenia projektowe

- Sygnał zegarowy: 50 MHz,
- Wyświetlane wyniki w zakresie: 0-59 minut i 0-59 sekund,
- Obsługa przycisków: start/pauza, zapisu wyniku, reset,
- Pamięć na maksymalnie trzy zapisane wyniki z flagami ważności,
- Testowanie przy użyciu przebiegów czasowych oraz zapisu wyników do pliku tekstowego

3. Schemat jednostki projektowej (entity)

4. Algorytm pracy układu

Poniższy schemat blokowy przedstawia algorytm działania stopera:



Opis algorytmu

1. Po sygnale resetu stoper wraca do stanu początkowego.
2. Po wciśnięciu przycisku start/pauza następuje:
    - Rozpoczęcie odliczania czasu (jeśli stoper był zatrzymany),
    - Wstrzymanie odliczania czasu (jeśli stoper był w trakcie działania),
    - Wznowienie odliczania czasu z pauzy
3. Wciśnięcie przycisku zapisu powoduje zapisanie bieżącego czasu do pamięci.
4. Wybranie pamięci (sygnał memory_select) powoduje wyświetlenie zapisanego wyniku.

5. Tabela sygnałów i zmiennych

Nazwa	Zastosowanie	Typ	Zakres wartości
clk	Sygnał zegarowy	STD_LOGIC	0, 1
rst	Reset układu	STD_LOGIC	0, 1
start_pause	Przycisk start/pauza	STD_LOGIC	0, 1
save	Przycisk zapisu	STD_LOGIC	0, 1
memory_select	Wybór pamięci	STD_LOGIC_VECTOR(1 downto 0)	"00", "01", "10", "11"
minutes	Wyświetlane minuty	INTEGER	0 - 59
seconds	Wyświetlane sekundy	INTEGER	0 - 59
memory_valid	Flagi ważności zapisanych czasów	STD_LOGIC_VECTOR(2 downto 0)	"000" - "111"

6. Listing kodu źródłowego modułu nadrzędnego

- Biblioteki i deklaracje:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

- Deklaracja entity:

```
entity Stoper is
    Port (
        clk           : in  STD_LOGIC;                    -- sygnał zegarowy (50MHz)
        rst           : in  STD_LOGIC;                    -- sygnał resetu
        start_pause   : in  STD_LOGIC;                    -- przycisk start/pauza
        save          : in  STD_LOGIC;                    -- przycisk do zapisu wyniku
        memory_select : in  STD_LOGIC_VECTOR(1 downto 0); -- wybór pamięci do wyświetlenia (2 bity)
        minutes       : out INTEGER range 0 to 59;        -- wyświetlane minuty
        seconds       : out INTEGER range 0 to 59;        -- wyświetlane sekundy
        memory_valid  : out STD_LOGIC_VECTOR(2 downto 0)  -- flagi ważności zapisanych czasów (czy czas jest aktualny i dobrze zapisany)
    );
end Stoper;
```

- Deklaracja architektury:

```
architecture Behavioral of Stoper is
```

- Stałe i typy:

```
-- stałe
constant CLK_FREQ : integer := 50_000_000; -- 50MHz (jedna pełna operacja zajmuje 20ns)

-- typ do przechowywania zapisanych czasów (struktura)
type stored_time is record
    minutes : integer range 0 to 59;
    seconds : integer range 0 to 59;
    valid   : std_logic;              -- flaga ważności danych
end record;

-- tablica do przechowywania zapisanych czasów (3 miejsca)
type memory_array is array (0 to 2) of stored_time;
```

- Sygnały wewnętrzne:

```
-- sygnały wewnętrzne
signal running       : std_logic := '0';                        -- status działania stopera
signal paused        : std_logic := '0';                        -- status pauzy
signal min_count     : integer range 0 to 59 := 0;              -- licznik minut
signal sec_count     : integer range 0 to 59 := 0;              -- licznik sekund
signal memory        : memory_array := (others => (0, 0, '0'));  -- pamięć czasów
signal memory_index  : integer range 0 to 2 := 0;               -- indeks do zapisu pamięci (wskazuje, która pamięć jest aktualnie używana do zapisu)
signal display_mins  : integer range 0 to 59 := 0;              -- minuty do wyświetlenia
signal display_secs  : integer range 0 to 59 := 0;              -- sekundy do wyświetlenia
```

- Główny proces (licznik i kontrola stopera):

```
-- główny proces sterujący działanie stopera
process(clk, rst)
    variable clk_counter : integer range 0 to CLK_FREQ-1 := 0; -- zlicznik taktów zegara (zmienna lokalna - aktualizuje sie natychmiast, liczy od 0 do 49_999_999)
begin
    if rst = '1' then
        -- reset wszystkich stanów
        min_count <= 0;
        sec_count <= 0;
        running <= '0';
        paused <= '0';
        clk_counter := 0;
    elsif rising_edge(clk) then
        -- obsługa przycisku start/pauza
        if start_pause = '1' then
            if running = '0' and paused = '0' then
                running <= '1';    -- rozpoczęcie działania stopera
            elsif running = '1' then
                running <= '0';    -- pauza
                paused <= '1';
            else
                running <= '1';    -- wznowienie z pauzy
                paused <= '0';
            end if;
        end if;

        -- zliczanie czasu gdy stoper działa
        if running = '1' then
            if clk_counter = CLK_FREQ-1 then
                clk_counter := 0;           -- reset licznika zegara
                if sec_count = 59 then
                    sec_count <= 0;
                    if min_count = 59 then
                        min_count <= 0;     -- przepełnienie minut
                    else
                        min_count <= min_count + 1;  -- inkrementacja minut (upływ 59 sekund)
                    end if;
                else
                    sec_count <= sec_count + 1;      -- inkrementacja sekund
                end if;
            else
                clk_counter := clk_counter + 1;      -- inkrementacja licznika zegara
            end if;
        end if;

        -- obsługa zapisu czasu
        if save = '1' and memory_index < 3 then
            memory(memory_index) <= (min_count, sec_count, '1'); -- zapis czasu do pamięci
            if memory_index < 2 then
                memory_index <= memory_index + 1; -- inkrementacja indeksu pamięci
            end if;
        end if;
    end if;
end process;
```

- Proces wyświetlanego czasu:

```
-- proces wyboru wyświetlanego czasu
process(memory_select, min_count, sec_count, memory)
begin
    case memory_select is
        when "00" =>   -- wyświetlanie bieżącego czasu
            display_mins <= min_count;
            display_secs <= sec_count;
        when "01" =>   -- wyświetlanie pierwszego zapisanego czasu
            display_mins <= memory(0).minutes;
            display_secs <= memory(0).seconds;
        when "10" =>   -- wyświetlanie drugiego zapisanego czasu
            display_mins <= memory(1).minutes;
            display_secs <= memory(1).seconds;
        when others => -- wyświetlanie trzeciego zapisanego czasu
            display_mins <= memory(2).minutes;
            display_secs <= memory(2).seconds;
    end case;
end process;
```

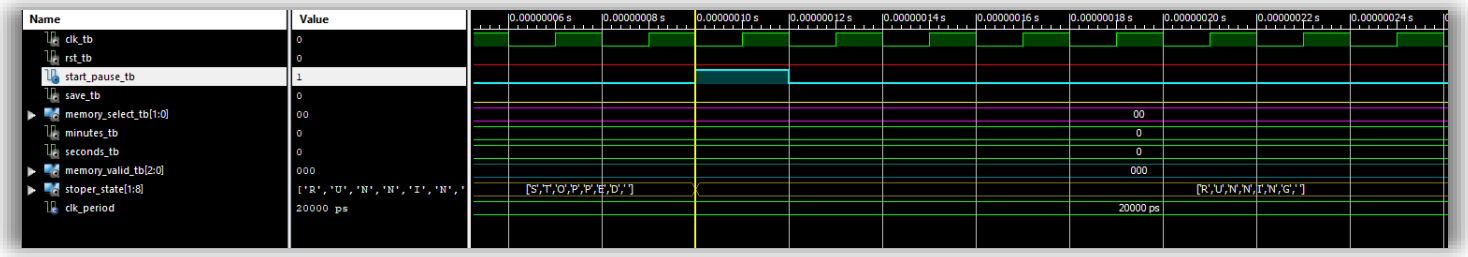
- Przypisanie wyjść:

```
-- przypisanie wyjść
minutes <= display_mins;
seconds <= display_secs;
-- flaga valid wskazuje, czy zapisany czas jest ważny
memory_valid <= memory(2).valid & memory(1).valid & memory(0).valid;
```

7. Wyniki symulacji

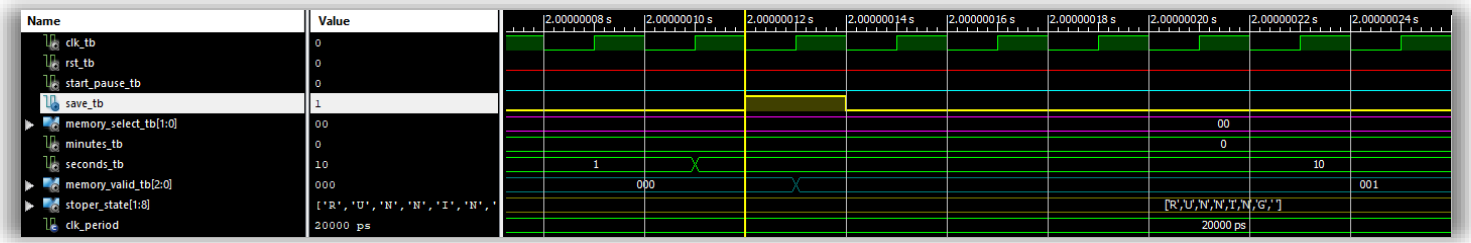
Poniższe zrzuty ekranu przedstawiają przebiegi czasowe z symulacji:

- **Sygnał start/wznowienie stopera:**

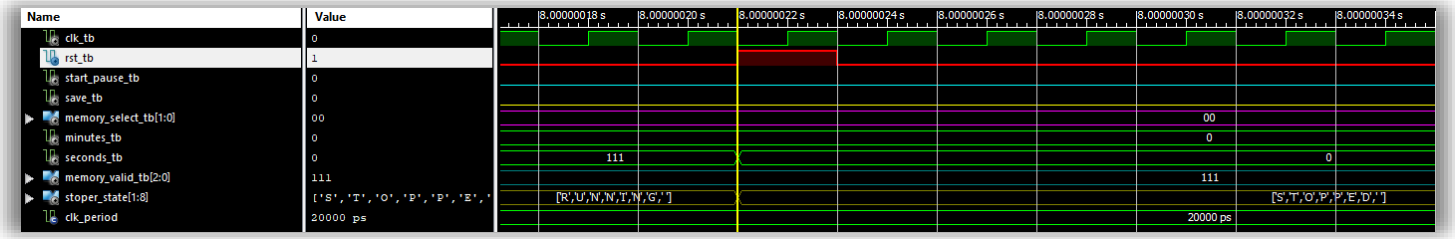


- **Sygnał zatrzymania stopera:**

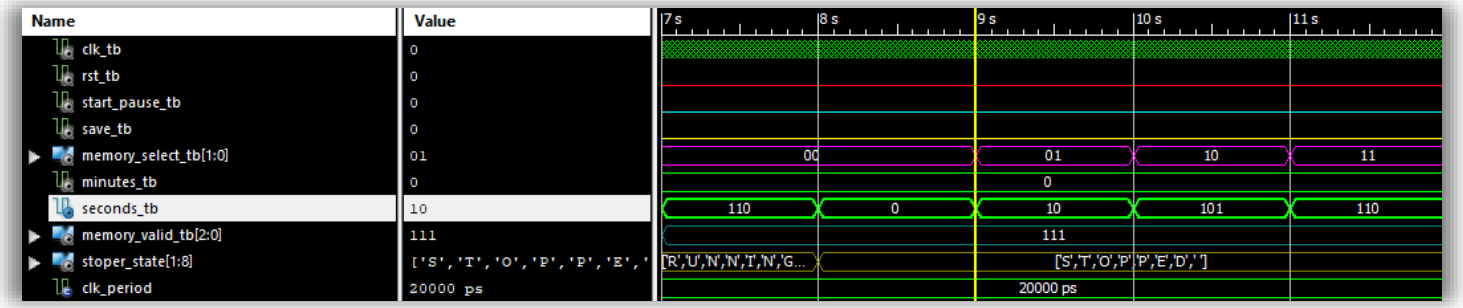
- **Sygnał zapisu wyników:**



- **Sygnał resetu stopera:**



- **Sygnał zapisania wyników w pliku tekstowym:**



8. **Wnioski**

Projekt stopera spełnia wszystkie założenia funkcjonalne:

- Zostało zaimplementowane poprawne odliczanie czasu z dokładnością do 1 sekundy.
- Funkcje startu/wznowienia, pauzy, zerowania (reset) oraz zapisu czasów działają zgodnie z oczekiwaniami.
- Pamięć umożliwia zapis do trzech wyników z ich poprawnym wyświetlaniem.

Możliwe usprawnienia projektu:

1. Rozszerzenie zakresu zapamiętywanych wyników.
2. Zmniejszenie opóźnień w obsłudze przycisków.

Podsumowując, projekt można uznać za zakończony sukcesem.